

example, the processor and processor features, types of devices, buses exported to the guest -- or it can be flexible, with the user getting a choice of selecting individual components and parameters.

There are some constraints to creating a virtualised environment. A set of sufficient requirements noted by Papek and Goldberg in their paper on virtual machine monitors are:

- **Fidelity:** Software that is running in a virtualised environment should not be able to detect that it is actually being run on a virtualised system.
- **Containment:** Activities within a virtual machine (VM) should be contained within the VM itself without disturbing the host system. A guest should not cause the host, or other guests running on the host, to malfunction.
- **Performance:** Performance is crucial to how the user sees the utility of the virtualising environment. In this age of extremely fast and affordable general-purpose computer systems, if it takes a few seconds for some input action to get registered in a guest, no one will be interested in using the virtual machine at all.
- **Stability:** The virtualisation software itself should be stable enough to handle the guest OS and any quirks it may exhibit.

There are several reasons why one would want virtualisation. For data centres, it makes sense to run multiple servers (Web, mail, etc) on a single machine. These servers are mostly under-utilised, so clubbing them on one machine with a VM for each of the existing machines, makes way for fewer machines, less rack-space and lower electricity consumption.

For enterprises, serving users' desktops on a VM simplifies management, IT servicing, security considerations and costs by reduced expenditure on desktops.

For developers, testing code written for different architectures or target systems becomes easier, since access to the actual system becomes optional. For example, a new mobile phone platform can be virtualised on a developer machine rather than actually deploying the software on the phone hardware each time, allowing for the software to be developed along with the hardware. The virtualised environment can also be used as validation for the hardware platform itself before going into production to avoid costs arising later due to changes that might be needed in the hardware.

There are several such examples that can be cited for any kind of application or use. It's not impossible to imagine a virtualised system being beneficial anywhere a computer is being used.

Now is a good time to get acquainted with some terms (the mandatory alphabet soup) that we'll be using throughout the article:

- **VM:** Virtual Machine
- **VMM:** Virtual Machine Monitor
- **Guest OS:** The OS that is run within a virtual machine
- **Host OS:** The OS that runs on the computer system
- **Paravirtualised guest:** The guest OS that is modified to have knowledge of a VMM
- **Full virtualisation:** The guest OS is run unmodified in this environment
- **Hypervisor:** An analogous term for a VMM
- **Hypercall:** The medium that a paravirtualised guest and the VMM communicate on

Types of VMM

There are several virtual machine monitors available. They differ in various aspects like scope, motivation, and method of implementation. A few types of monitor software are:

- **'Native' hypervisors:** These VMMs have an OS associated with them. A complete software-based implementation will need a scheduler, a memory management subsystem and an IO device model to be exported to the guest OS. Examples are VMWare ESX server, Xen, KVM, and IBM mainframes. In IBM mainframes, the VMM is an inherent part of the architecture.
- **Containers:** In this type of virtualisation, the guest OS and the host OS share the same kernel. Different namespaces are allocated for different guests. For example, the process identifiers, file descriptors, etc, are 'virtualised', in the sense a PID obtained for a process in the guest OS will only be valid within that guest. The guest can have a different userland (for example, a different distribution) from the host. Examples are OpenVZ, FreeVPS, and Linux-Vserver.
- **Emulation:** Each instruction in the guest is emulated. It is possible to run code compiled for different architectures on a computer, like running ARM code on a PowerPC machine. Examples are qemu, pearpc, etc. qemu supports multiple CPU types, and runs ARM code under x86 as well as x86 under x86, whereas pearpc only emulates the PPC platform.

Hardware support on x86

Virtualising the x86 architecture is difficult since the instruction and register sets are not compatible with virtualisation. Not all access to privileged instructions or registers raises a trap. So we either have to emulate the guest entirely, or patch it at run-time to behave in a particular way.

With the two leading x86 processor manufacturers, Intel and AMD, adding virtualisation extensions to their processors, virtualising the x86 platform seamlessly has become easier. The ideas in their virtualisation extensions are more or less the same, with the implementation, instructions and register sets being